BTS SIO — Option SLAM

Services Informatiques aux Organisations

RAPPORT DE STAGE

Développement de MakeBetterBot 2.0.0 Beta

Refonte et développement d'un bot d'automatisation Discord

Année scolaire 2024–2025

Introduction

Dans le cadre de ma formation en BTS SIO, option SLAM (Solutions Logicielles et Applications Métiers), j'ai effectué mon stage au sein d'une agence spécialisée dans la création, l'automatisation et la gestion de communautés Discord. Cette structure, positionnée dans un secteur en pleine expansion, développe des solutions logicielles sur mesure destinées à optimiser l'expérience utilisateur et l'administration de serveurs communautaires hébergés sur la plateforme Discord.

Discord, initialement conçu pour les communautés de joueurs, s'est progressivement imposé comme une plateforme de communication incontournable pour des usages professionnels, éducatifs et communautaires. Pour répondre aux besoins croissants de ses clients, l'agence a entrepris une refonte totale de son principal outil d'automatisation : MakeBetterBot. L'objectif était de passer d'une architecture vieillissante à une solution moderne, maintenable et extensible, désignée sous le nom de version 2.0.0 Beta.

C'est dans ce contexte que j'ai intégré l'équipe de développement en tant que développeur logiciel stagiaire. Mon rôle a consisté à contribuer activement à cette refonte, en prenant en charge plusieurs missions techniques couvrant un large spectre du développement applicatif : conception de commandes interactives, modélisation de bases de données, internationalisation, migration de code existant et travail collaboratif via des outils de versioning professionnels.

Ce rapport présente successivement l'entreprise qui m'a accueilli, les différentes missions que j'ai réalisées, les solutions techniques mises en œuvre ainsi que le bilan de cette expérience professionnelle.

Présentation de l'Entreprise

Activité et positionnement

L'entreprise est une agence spécialisée dans l'écosystème Discord, offrant à ses clients un triptyque de services complémentaires : la création de serveurs communautaires sur mesure, l'automatisation des processus de gestion via des bots logiciels, et l'administration continue des communautés ainsi constituées. Son positionnement répond à une demande croissante des organisations souhaitant déléguer la gestion technique et humaine de leurs espaces de communication en ligne.

La valeur ajoutée de l'agence repose principalement sur son expertise en développement de bots Discord, des programmes autonomes capables d'interagir avec les utilisateurs, de modérer les échanges, d'automatiser des tâches répétitives et d'enrichir l'expérience communautaire grâce à des fonctionnalités métier spécifiques.

Contexte du projet : MakeBetterBot 2.0.0 Beta

Au cœur de l'activité technique de l'agence se trouve MakeBetterBot, un bot d'automatisation propriétaire déployé sur l'ensemble des serveurs clients. Face à l'accumulation d'une dette technique significative dans les versions antérieures — code peu structuré, absence de modèles de données normalisés, multilinguisme non pris en charge — la direction technique a pris la décision stratégique d'opérer une refonte complète du produit.

La version 2.0.0 Beta représente ainsi une rupture architecturale avec les versions précédentes. Elle repose sur un socle technique moderne, standardisé et documenté, conçu pour accueillir de nouvelles fonctionnalités tout en garantissant la maintenabilité à long terme du code source.

Environnement technique

La stack technologique retenue pour cette refonte reflète les standards actuels du développement d'applications back-end événementielles :

Back-end	Node.js — JavaScript (runtime asynchrone adapté aux I/O intensifs)
API Discord	Discord API v2 avec Display Components, ContainerBuilder, TextDisplay
Base de données	MongoDB (NoSQL) avec l'ODM Mongoose pour la validation de schémas
Configuration	Fichiers de configuration et traductions au format JSON

Versioning	Git avec GitHub — Issues, Pull Requests, Conventional Commits

Le choix de MongoDB, couplé à Mongoose, mérite une attention particulière. Si MongoDB est une base de données orientée documents — donc schéma-libre par nature — l'utilisation de Mongoose impose au niveau applicatif des schémas stricts qui structurent et valident les données avant leur persistance. Cette approche permet de bénéficier de la flexibilité du NoSQL tout en garantissant l'intégrité des données, caractéristique habituellement propre aux bases relationnelles SQL.

Configuration de l'environnement de développement

La première semaine de stage a été consacrée à la mise en place de l'environnement de développement local. Cela a impliqué la configuration du fichier .env, regroupant l'ensemble des variables sensibles nécessaires au lancement du bot.

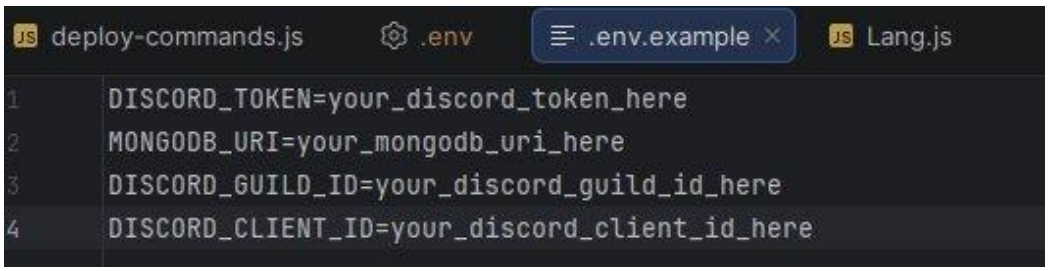

Figure 1 — Structure du fichier .env.example définissant les variables d'environnement requises

La connexion à la base de données a été assurée via MongoDB Atlas, la solution cloud managée de MongoDB, qui m'a fourni une instance distante et persistante pour mes tests. La chaîne de connexion SRV générée par Atlas a été intégrée dans la variable MONGODB_URI du fichier .env.

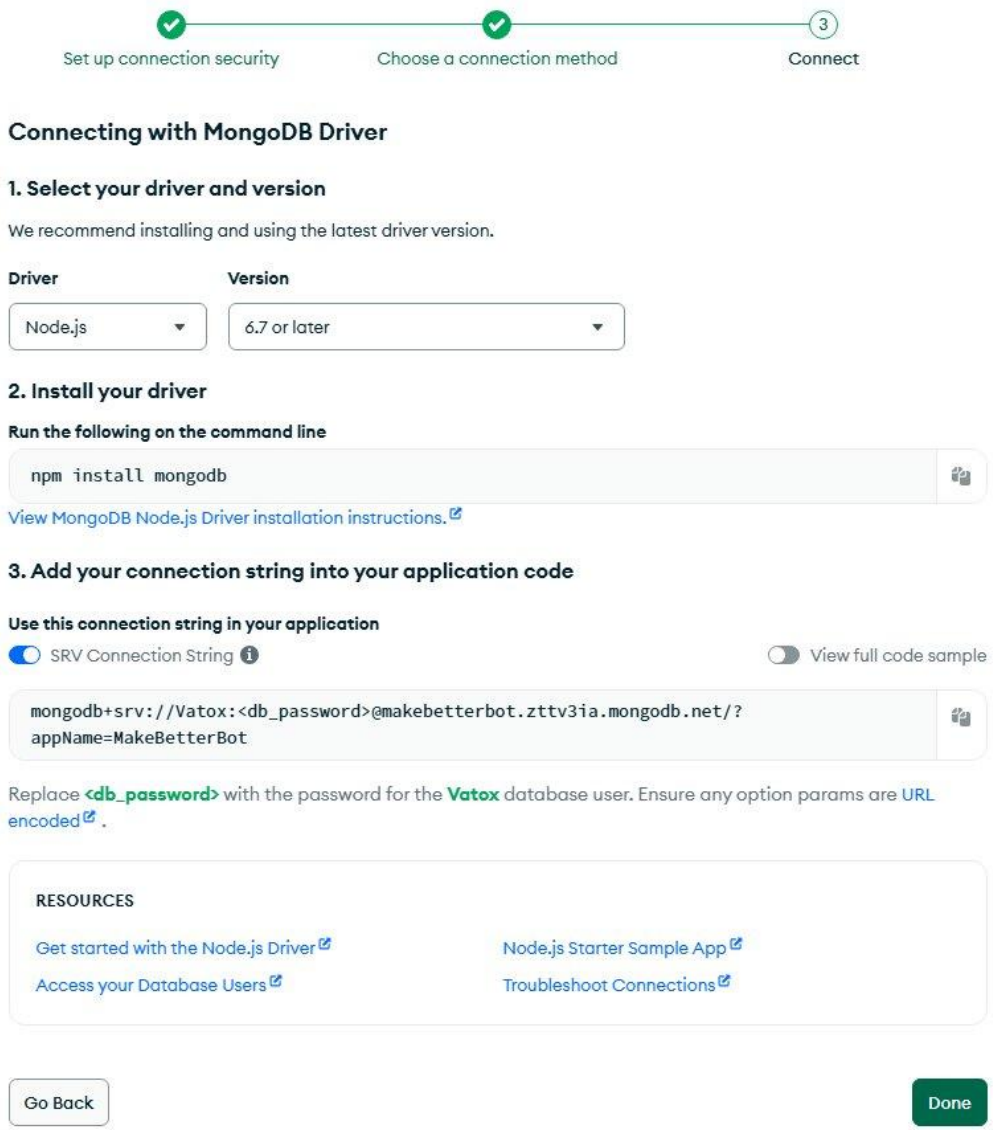

Figure 2 — Interface MongoDB Atlas : génération de la chaîne de connexion Node.js (Driver 6.7+)

Missions Réalisées

Organisation du travail

L'intégralité des missions a été planifiée et suivie via GitHub Issues et Pull Requests. Chaque fonctionnalité correspond à une issue décrivant le besoin fonctionnel, liée à une Pull Request contenant le code correspondant. Ce workflow garantit une traçabilité complète entre les exigences et le code livré.

Mission 1 — Développement de la commande /builder

Contexte et enjeux

La commande /builder existait dans une version fonctionnelle mais présentait tous les signes caractéristiques du "spaghetti code" : absence de séparation des responsabilités, logique de gestion des états mélangée à la logique d'affichage, et incompatibilité avec les nouvelles fonctionnalités de l'API Discord. Ma mission a consisté en une réécriture complète de cette commande.

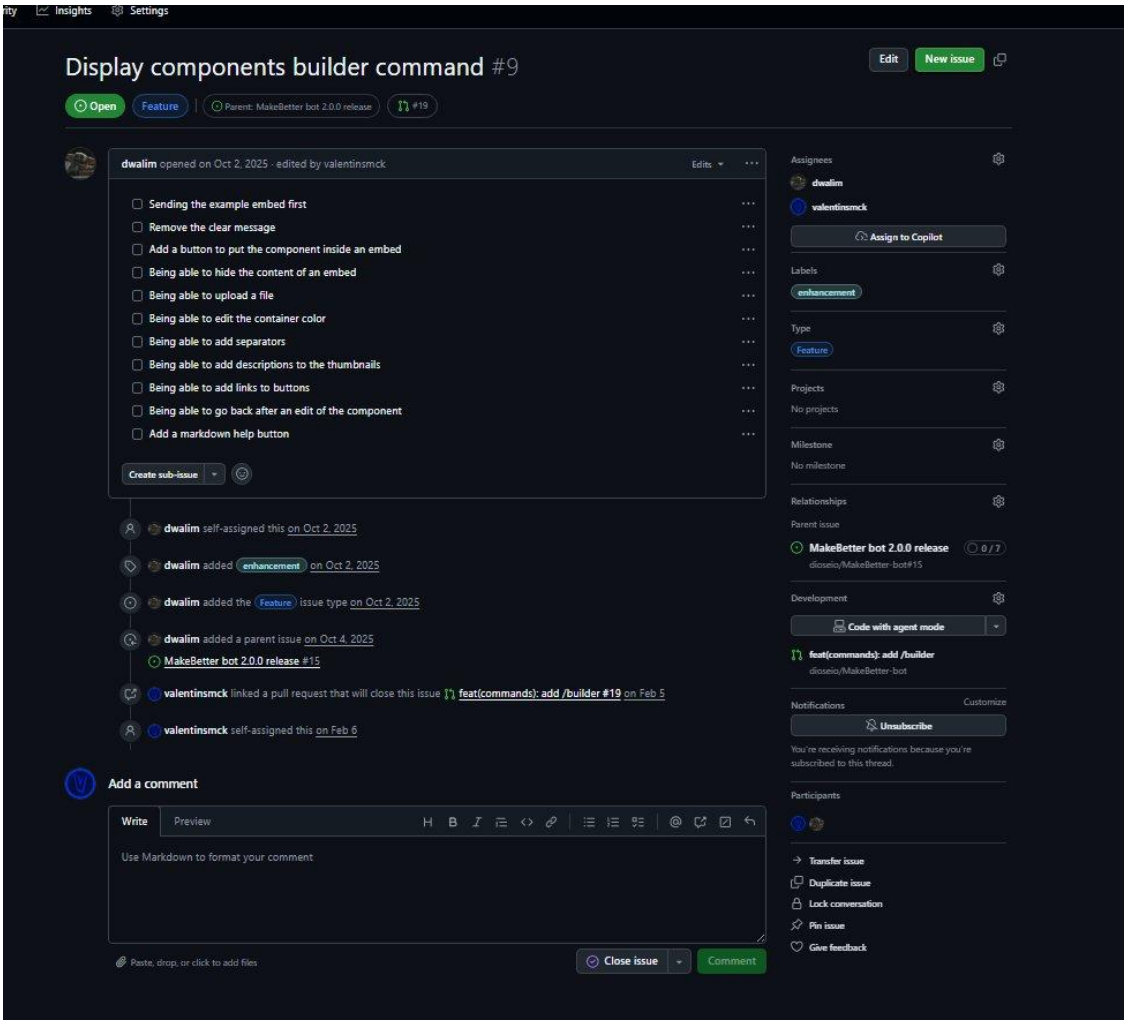

Figure 3 — Issue GitHub #9 : liste des fonctionnalités attendues pour la commande /builder

Réalisation technique

La nouvelle implémentation repose sur les Display Components V2 de Discord, une API expérimentale permettant de construire des interfaces riches et interactives directement dans l'environnement Discord. J'ai utilisé les classes ContainerBuilder et TextDisplay pour structurer les messages de manière modulaire.

L'interaction utilisateur est gérée via des Modals — des formulaires Discord affichés en fenêtre surgissante — permettant une édition en temps réel des différents composants d'un embed. Un soin particulier a été apporté à la persistance des états : à chaque interaction, l'état courant de l'embed en construction est conservé et rechargé, évitant toute perte de données lors des transitions entre étapes. La gestion des médias a été améliorée avec la prise en charge d'une galerie multi-images et d'un bouton Spoiler.

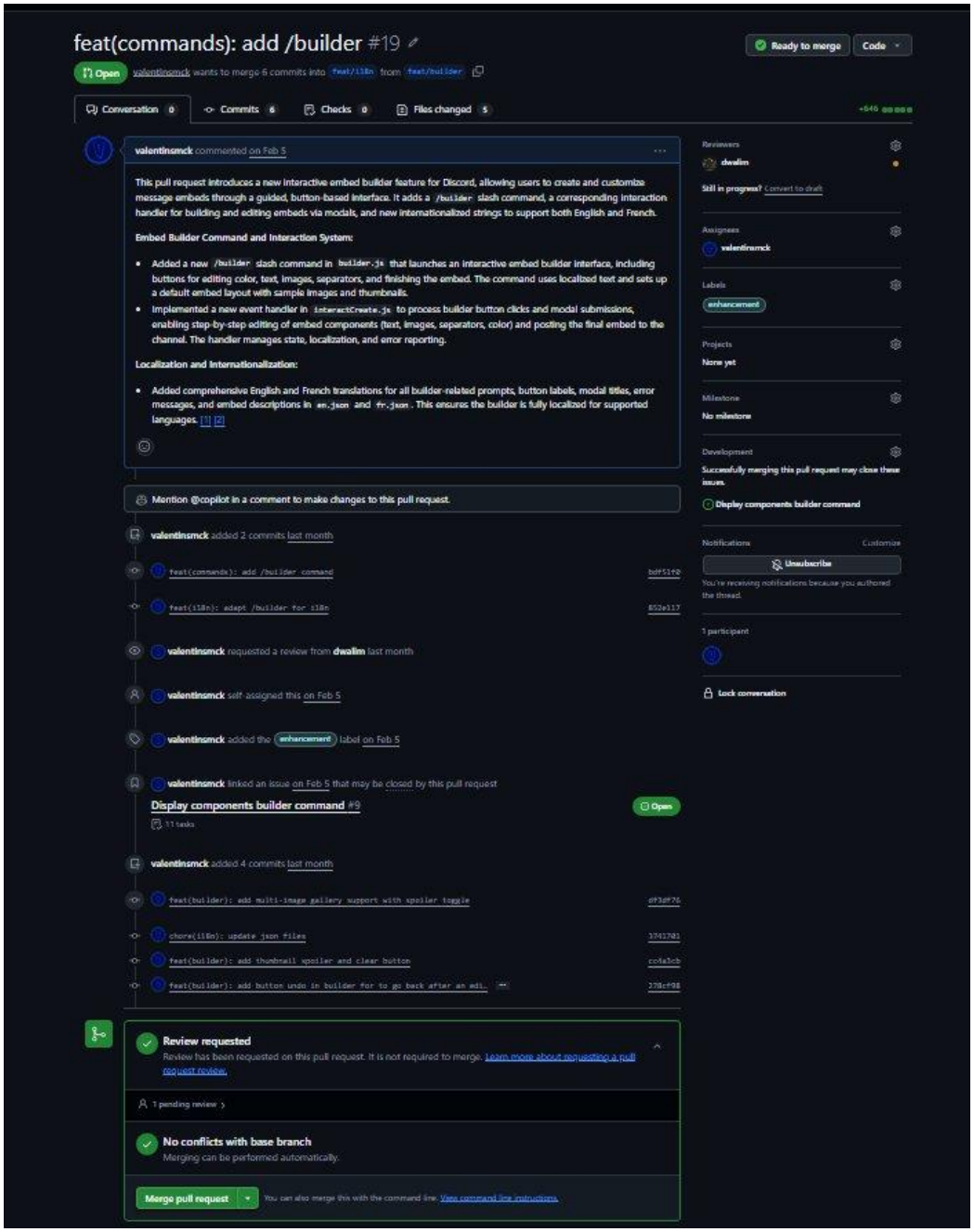

Figure 4 — Pull Request #19 : feat(commands): add /builder — description des changements apportés

Compétences mobilisées

- Programmation événementielle asynchrone (Node.js / JavaScript)

- Manipulation de l'API Discord v2 et de ses composants expérimentaux (ContainerBuilder, TextDisplay)
- Conception d'interfaces utilisateur interactives via les Modals Discord
- Gestion des états applicatifs lors d'interactions multi-étapes

Mission 2 — Modélisation de la base de données

Contexte et enjeux

L'absence de modèles de données formalisés dans les versions antérieures du bot constituait une source majeure d'instabilité. Des données mal formatées ou incomplètes pouvaient être insérées en base sans aucune validation, provoquant des erreurs en cascade difficiles à diagnostiquer.

Réalisation technique

J'ai conçu et implémenté l'ensemble des modèles Mongoose nécessaires aux nouvelles fonctionnalités. Chaque modèle définit explicitement les champs attendus, leurs types, leurs contraintes (required, unique, default) ainsi que les relations inter-collections.

```
deploy-commands.js   .env   Lang.js

1  const { Schema, model :{...} } = require('mongoose');
2
3  const LangSchema :Schema<any, Model<...>, {...}, {...}, {...}, {...}, DefaultSchemaOptions, {...}> = new Schema( definition: {
4      guildId: { type: String, required: true, unique: true },
5      language: { type: String, default: 'en' }
6  });
7
8  module.exports = model('Lang', LangSchema);
```

Figure 5 — Modèle Mongoose Lang.js : association guildId / langue avec contrainte d'unicité

Ce modèle associe un identifiant de serveur Discord (guildId) à une préférence de langue, avec une valeur par défaut en anglais. La contrainte unique: true empêche la création de doublons pour un même serveur, garantie critique pour le bon fonctionnement du système i18n. Cette approche — ODM strict sur base NoSQL — offre la scalabilité de MongoDB tout en imposant la rigueur de schémas typés propre aux bases relationnelles.

Compétences mobilisées

- Conception de schémas de données avec Mongoose
- Gestion des contraintes d'intégrité (unicité, valeurs obligatoires, valeurs par défaut)
- Modélisation des relations entre collections NoSQL
- Compréhension des compromis entre bases relationnelles et documents

Mission 3 — Internationalisation (i18n)

Contexte et enjeux

MakeBetterBot étant déployé sur des serveurs de communautés francophones et anglophones, le support multilingue représentait une fonctionnalité prioritaire. Il s'agissait de permettre à chaque serveur de définir sa langue de prédilection, afin que toutes les réponses du bot soient présentées dans la langue appropriée.

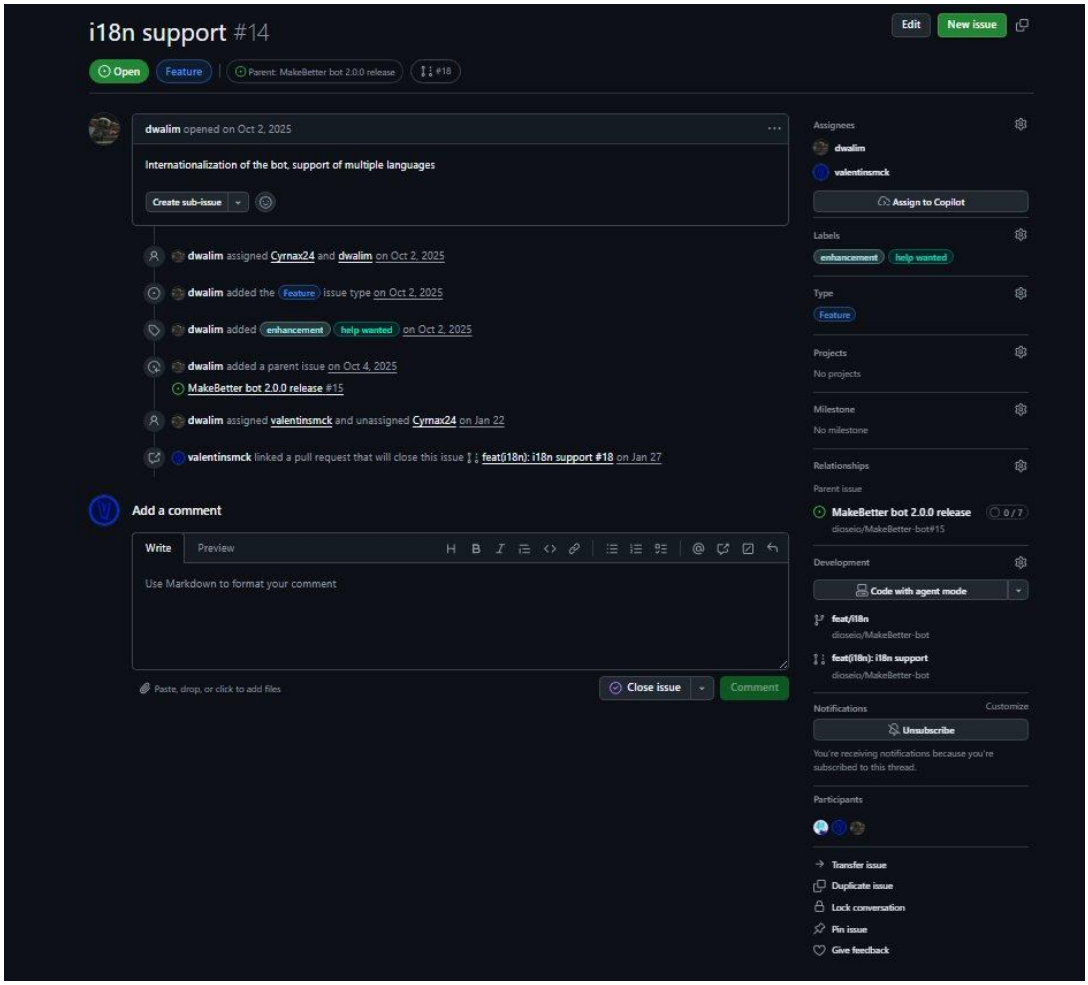

Figure 6 — Issue GitHub #14 : cahier des charges de la fonctionnalité i18n

Réalisation technique

J'ai mis en place un système d'internationalisation reposant sur des fichiers de traduction JSON organisés par langue. Une fonction utilitaire de traduction (t) résout dynamiquement les chaînes localisées en fonction de la langue du serveur, récupérée depuis la base de données via le modèle Lang. Les commandes Discord ont été mises à jour pour utiliser setNameLocalizations et setDescriptionLocalizations, affichant des noms et descriptions localisés directement dans l'interface Discord.

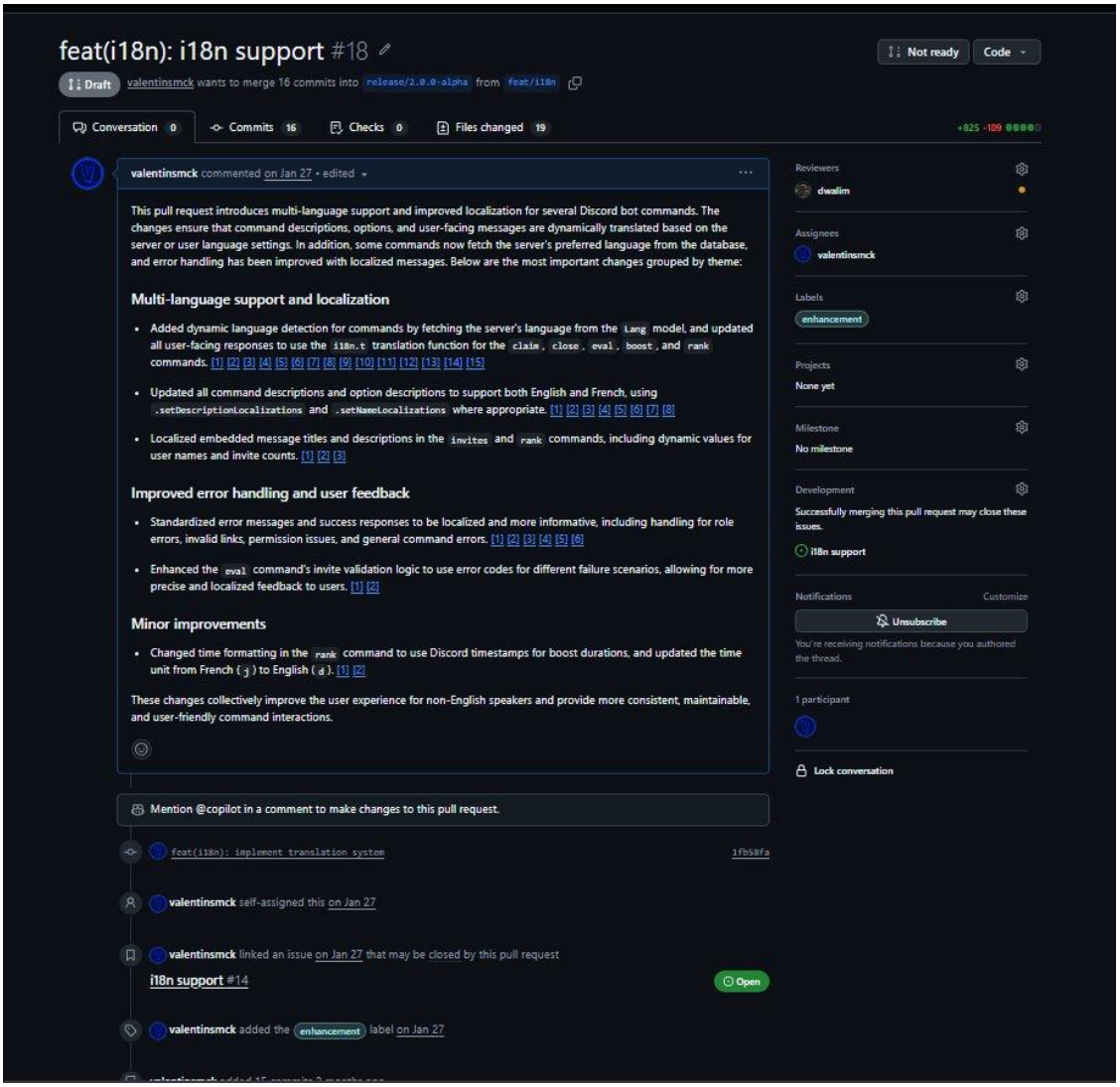

Figure 7 — Pull Request #18 : feat(i18n): i18n support — détail des modifications apportées

Compétences mobilisées

- Architecture d'un système i18n en JavaScript
- Manipulation de fichiers JSON comme source de configuration et de données
- Intégration avec l'API Discord pour la localisation des commandes
- Conception de messages d'erreur explicites et localisés

Mission 4 — Refactoring et transfert du code legacy

Contexte et enjeux

Parallèlement au développement de nouvelles fonctionnalités, il était nécessaire de migrer les commandes fonctionnelles des anciens bots vers la nouvelle architecture partagée de la v2.0.0 Beta. Cet exercice de refactoring ne se limitait pas à une copie de code, mais impliquait une réécriture partielle pour respecter les nouvelles conventions architecturales.

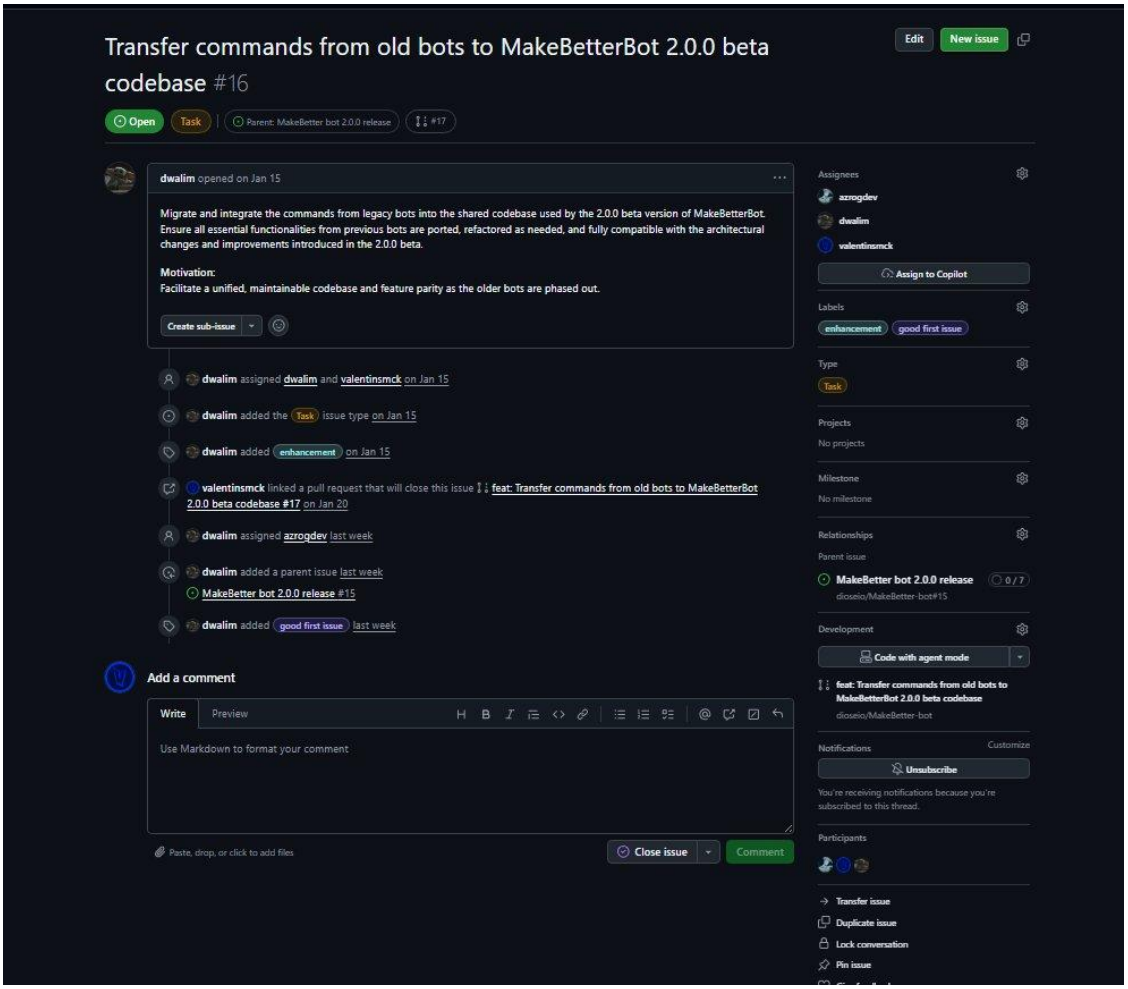

Figure 8 — Issue GitHub #16 : description de la mission de migration des commandes legacy

Réalisation technique

La migration a concerné un large ensemble de commandes couvrant plusieurs domaines fonctionnels : gestion d'événements Halloween, système de niveaux d'expérience (XP), commandes d'évaluation et de tickets, gestion des invitations et commandes utilitaires diverses.

Pour chaque commande migrée, j'ai appliqué les principes suivants : respect de la nouvelle structure de fichiers, remplacement des anciens gestionnaires d'événements, adaptation aux

schémas Mongoose, et intégration du système i18n. La Pull Request #17 ci-dessous synthétise l'ensemble du travail réalisé dans cette mission.

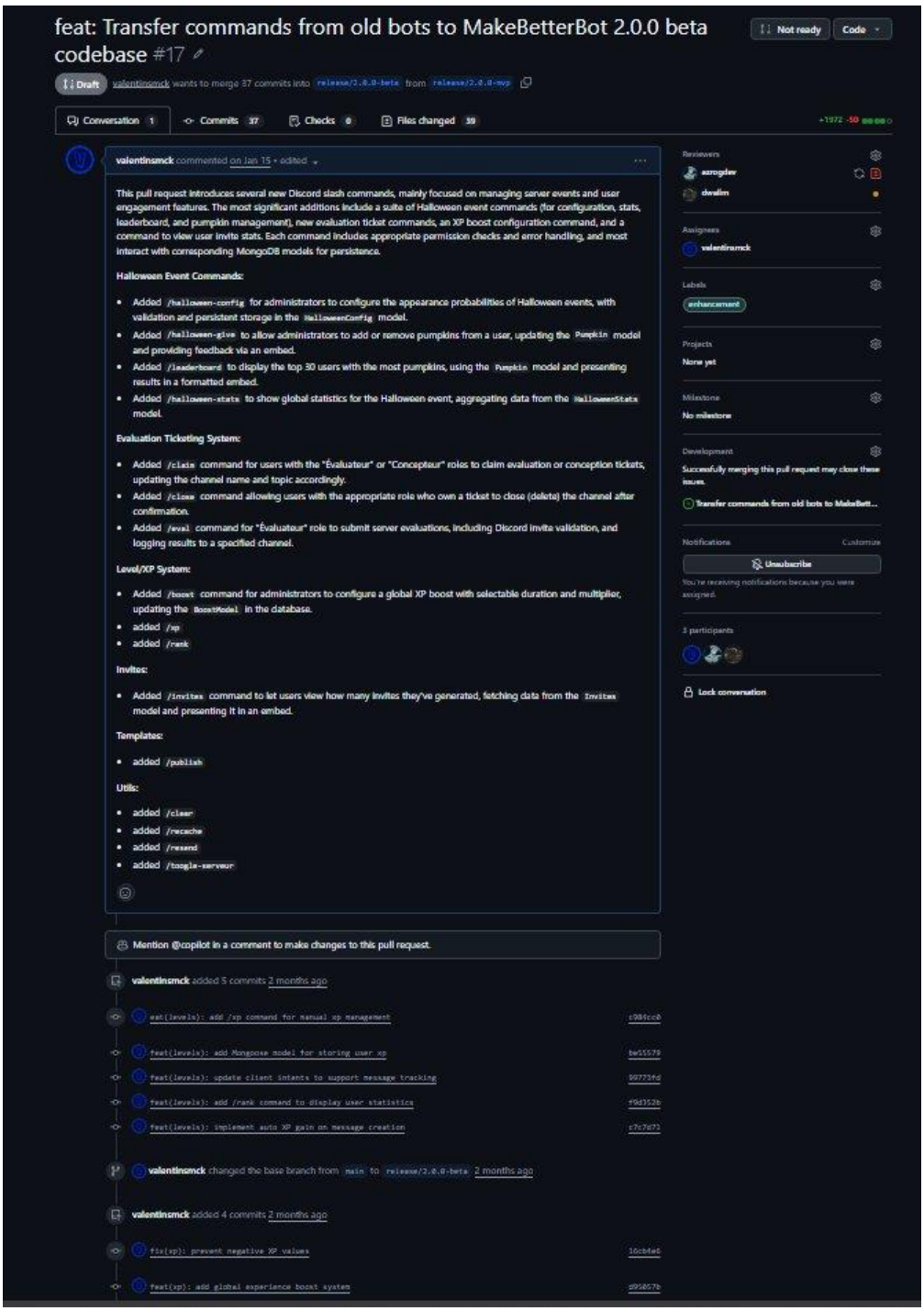

Figure 9 — Pull Request #17 (1/2) : description des commandes migrées depuis les anciens bots

Figure 10 — Pull Request #17 (2/2) : historique des commits de migration

Compétences mobilisées

- Analyse et compréhension de code existant (code review, dette technique)
- Refactoring méthodique en conservant les comportements fonctionnels
- Harmonisation du code avec une architecture cible
- Résilience face à des bases de code non documentées

Mission 5 — Gestion de projet et travail collaboratif

Contexte et enjeux

Bien que transversale aux autres missions, la dimension collaborative mérite d'être traitée comme une mission à part entière, car elle a structuré l'ensemble de mon travail quotidien et reflète des pratiques professionnelles directement valorisables dans le monde du développement logiciel.

Réalisation technique

L'intégralité de mon travail a été versionné sur un dépôt GitHub distant, en suivant un workflow de branches structuré (feature branches, release branches). Chaque nouvelle fonctionnalité a fait l'objet d'une Pull Request dédiée, accompagnée d'une description des changements, permettant une revue de code par les membres de l'équipe.

Le suivi des tâches a été assuré via le système d'Issues GitHub : chaque issue décrit un besoin fonctionnel ou une correction, et est liée à la Pull Request qui la résout. J'ai également appliqué la convention Conventional Commits pour la rédaction de mes messages de commit (feat, fix, refactor, chore...), facilitant la lecture de l'historique et la génération de changelogs.

Compétences mobilisées

- Utilisation avancée de Git : branches, merges, gestion des conflits
- Travail collaboratif sur GitHub : Issues, Pull Requests, revue de code
- Application de la norme Conventional Commits
- Communication technique via des descriptions de PR structurées

Conclusion

Ce stage au sein de l'agence m'a offert une immersion complète dans les réalités du développement logiciel professionnel. À travers les cinq missions réalisées sur le projet MakeBetterBot 2.0.0 Beta, j'ai pu mobiliser et approfondir un large panel de compétences directement ancrées dans le référentiel du BTS SIO SLAM.

Sur le plan technique, j'ai eu l'occasion de travailler avec un stack moderne et cohérent — Node.js, MongoDB, Mongoose, API Discord v2 — dans un contexte de production réel. La conception de commandes interactives exploitant des fonctionnalités expérimentales de l'API Discord m'a confronté à des problématiques concrètes de gestion d'états, de persistance et d'expérience utilisateur. La modélisation de la base de données m'a permis d'appréhender les compromis architecturaux entre les paradigmes NoSQL et relationnel. L'implémentation du système d'internationalisation a renforcé ma compréhension de la structuration des données de configuration et de leur utilisation dynamique en cours d'exécution.

La mission de refactoring du code legacy a été particulièrement formatrice : elle m'a appris à lire et comprendre un code que je n'avais pas écrit, à identifier et traiter la dette technique, et à adapter des fonctionnalités existantes à une architecture cible sans rompre les comportements attendus. Ces compétences sont essentielles dans un contexte professionnel où la maintenance et l'évolution de systèmes existants représentent une part significative du travail de développeur.

Sur le plan méthodologique, l'utilisation quotidienne de GitHub — avec ses Issues, ses Pull Requests et la convention Conventional Commits — m'a initié aux pratiques de développement collaboratif en vigueur dans l'industrie. J'ai compris que la qualité d'un logiciel ne se mesure pas uniquement à ses fonctionnalités, mais aussi à la lisibilité de son historique, à la clarté de sa documentation et à la rigueur de son processus de livraison.

Cette expérience confirme mon attrait pour le développement back-end et les architectures applicatives. Elle m'a également donné une vision plus précise des compétences à développer dans la suite de mon parcours, notamment en matière de conception d'API RESTful, de tests automatisés et de déploiement continu. Je repars de ce stage avec la conviction que le développement logiciel est un artisanat exigeant, où la rigueur technique et la capacité à travailler en équipe sont tout aussi déterminantes que la maîtrise des langages de programmation.